Digital Logic - Chapter 2

ECU 2026 - Made By Dawood

1 Part 1: Boolean Algebra

What is Boolean Algebra?

A mathematical method for dealing with values that are only:

- **0 = False** (غلط)
- **1 = True** (صح)

We use two fundamental operations:

- **+ (OR)** → "أو"
- **· (AND)** → "و"
- **′ (NOT)** → العكس

أي متغير زي **x** أو **y** بيكون يا **0** يا **1**، وبنطبق عليهم قوانين زي الحساب العادي بس مختلفة شوية.

القوانين الـ 6 (Huntington Postulates)

1. Closure (الانغلاق)

لو دخلت قيم من **B = {0,1}**

الناتج لازم يفضل برضه 0 أو 1

✓ مثال:

$$1 + 0 = 1 \checkmark$$
$$1 \cdot 1 = 1 \checkmark$$

2. Associative (التجميع)

مفيش فرق نحسب الأول مين

$$(x + y) + z = x + (y + z)$$
$$(x \cdot y) \cdot z = x \cdot (y \cdot z)$$

✓ يعني الأقواس مش بتفرق

3. Commutative (التبديل)

ممكن تبدل الأماكن عادي

$$x + y = y + x$$
$$x \cdot y = y \cdot x$$

✓ زي الجمع والضرب العادي

4. Identity (العنصر المحايد)

في عنصر لما تستخدمه مش بيغير القيمة

$$x + 0 = x$$
$$x \cdot 1 = x$$

✓ 0 محايد في OR

✓ 1 محايد في AND

5. Complement (المتمم)

كل قيمة ليها عكس

$$x + x' = 1$$
$$x \cdot x' = 0$$

✓ لو عندك 1 → العكس 0

✓ لو 0 → العكس 1

6. Distributive (التوزيع)

زي التوزيع في الرياضيات بس هنا الاتنين موجودين

$$x \cdot (y + z) = x \cdot y + x \cdot z$$
$$x + (y \cdot z) = (x + y)(x + z)$$

🔁 Duality Principle

أي قانون ينفع تقلب فيه: **OR ↔ AND** و **1 ↔ 0** ويفضل صح.

$$x + 0 = x \quad \xrightarrow{\text{dual}} \quad x \cdot 1 = x$$

⚡ Relation to Logic Gates

- **AND** → AND Gate
- **OR** → OR Gate
- **NOT (')** → Inverter

2 Part 2: Basic Theorems & Properties

📖 All Theorems

🟢 1. Identity

$$x + 0 = x \qquad x \cdot 1 = x$$

0 في **OR** ملوش تأثير | **1** في **AND** ملوش تأثير

🔴 2. Complement

$$x + x' = 1 \qquad x \cdot x' = 0$$

أي حاجة **+** عكسها **1 =** | أي حاجة **×** عكسها **0 =**

🟠 3. Idempotent

$$x + x = x \qquad x \cdot x = x$$

لو كررت نفس المتغير → مفيش تغيير

🟣 4. Dominance (Theorem 2)

$$x + 1 = 1 \qquad x \cdot 0 = 0$$

OR مع **1** → دايمًا **1** | **AND** مع **0** → دايمًا **0**

🟢 5. Involution

$$(x')' = x$$

عملت **NOT** مرتين → رجعت للأصل

🟠 6. Commutative

$$x + y = y + x \qquad xy = yx$$

7. Associative

$$x + (y + z) = (x + y) + z \qquad x(yz) = (xy)z$$

الأقواس مش مهمة هنا

8. Distributive

$$x(y + z) = xy + xz$$
$$x + yz = (x + y)(x + z)$$

دي مهمة جدًا في التبسيط

9. DeMorgan's Theorem

$$(x + y)' = x' \cdot y'$$
$$(xy)' = x' + y'$$

الفكره: غيّر العملية (+ ↔ ·) وخد **NOT** لكل حاجة

10. Absorption

$$x + xy = x$$
$$x(x + y) = x$$

دي بتحذف جزء كبير من المسألة

ترتيب العمليات الحسابية (مهم في الامتحان):

1. Parentheses ( )
2. NOT (')
3. AND (·)
4. OR (+)

3 Part 3: Truth Tables

What is a Truth Table?

ال **Truth Table** هو جدول بيجرب كل الاحتمالات الممكنة للمدخلات **(x, y, ...)** ويشوف الناتج بيطلع إيه.

Number of Rows

$$\text{Number of rows} = 2^n$$

Where n = number of variables. Example: 2 variables $\rightarrow 2^2 = 4$ rows.

Base Table (2 variables)

x	y
0	0
0	1
1	0
1	1

Operations in Truth Tables

AND (·)

لازم الاتنين = **1** علشان الناتج = **1**

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

OR (+)

لو واحد بس = **1** → الناتج = **1**

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

NOT (')

بيقلب القيمة

x	x'
0	1
1	0

Example: Proving DeMorgan

$$(x + y)' = x'y'$$

Step 1: Compute $x + y$

x	y	x+y	(x+y)'
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Step 2: Compute x' , y' , and $x'y'$

x	x'	y	y'	x'y'
0	1	0	1	1
0	1	1	0	0
1	0	0	1	0
1	0	1	0	0

✅ Both columns give **1 0 0 0** → Law is verified!

Example: Absorption $x + xy = x$

x	y	xy	x + xy	x (for comparison)
0	0	0	0	0
0	1	0	0	0
1	0	0	1	1
1	1	1	1	1

✅ Identical columns → Law verified!

4 Part 4: Boolean Functions

What is a Boolean Function?

دي معادلة بتاخد قيم **x, y, z** (كل واحد يا **0** يا **1**) وتطلع ناتج واحد (**0** أو **1**).

$$F = y'z + x$$

⊞ Step 1 – All Combinations (3 variables → 8 rows)

x	y	z
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1

1	1	0
1	1	1

Step 2 – Build Column by Column

نفكك المعادلة: y' ← نقلب y، ثم y'z ← AND، ثم F = y'z + x ← OR

Final Truth Table for $F = y'z + x$

x	y	z	y'	y'z	F
0	0	0	1	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	1	0	0	1

من المعادلة:

- لو x = 1 ← F = 1
- أو لو (y = 0 و z = 1)

5 Part 5: Boolean Simplification

Key Laws Used in Simplification

Complement

$$x \cdot x' = 0 \quad x + x' = 1$$

Identity

$$x + 0 = x \quad x \cdot 1 = x$$

Null (Dominance)

$$x \cdot 0 = 0 \quad x + 1 = 1$$

Absorption

$$x + xy = x \quad x(x + y) = x$$

Distributive

$$x(y + z) = xy + xz$$

$$x + yz = (x + y)(x + z)$$

Commutative

$$x + y = y + x \quad x \cdot y = y \cdot x$$

Associative

$$(x + y) + z = x + (y + z)$$

$$(x \cdot y) \cdot z = x \cdot (y \cdot z)$$

DeMorgan's Theorems

$$(X \cdot Y)' = X' + Y'$$

$$(X + Y)' = X' \cdot Y'$$

Double Negation

$$(X')' = X$$

Idempotent

$$x + x = x \quad x \cdot x = x$$

Worked Examples

Example 1: $x(x' + y)$

1. Distribute: $= xx' + xy$
2. Apply $xx' = 0$: $= 0 + xy$
3. Result: $= xy$

> ✓ When you see $x \cdot x'$ → eliminate it immediately = 0

Example 2: $x + x'y$

1. Use distributive form: $= (x + x')(x + y)$
2. Apply $x + x' = 1$: $= 1 \cdot (x + y)$
3. Result: $= x + y$

Example 3: $(x + y)(x + y')$

1. Expand: $= x + xy' + xy + yy'$
2. Apply $yy' = 0$: $= x + xy' + xy$
3. Factor x: $= x(1 + y' + y)$
4. Apply $y + y' = 1$: $= x(1 + 1) = x$

6 Part 6: DeMorgan's Theorem

Rule

+ Case 1 – OR

$$(x + y)' = x' \cdot y'$$

OR (+) اتحولت ← AND (·)
وكل متغير اتقلب

× Case 2 – AND

$$(xy)' = x' + y'$$

AND اتحولت ← OR وكل
متغير اتقلب

Quick Reference

Before	After
+	·
·	+

x	x'

أمثلة تطبيقية (DeMorgan)

مثال

السؤال: $(x+y)'$

الحل:

- اكسر القوس
- اقلب $\cdot \rightarrow +$
- اقلب كل متغير

$$= x'y'$$

مثال تاني

السؤال: $(xy)'$

الحل:

$$= x' + y'$$

مثال أكبر شوية

السؤال: $(A+B+C)'$

الخطوات:

- اكسر القوس
- اقلب (·) AND $\rightarrow$ +
- كل واحد يتحول لـ '

الناتج:

$$= A' \cdot B' \cdot C'$$

مثال العكس

السؤال: $(ABC)'$

الحل:

$$= A' + B' + C'$$

أهم فكرة تفهمها

- الـ **NOT (')** بيوزع على كل حاجة جوه.
- بس وهو بيوزع: **بيقلب العمليات** (الجمع يبقى ضرب، والضرب يبقى جمع).

7 Part 7: Complement of a Function

⇄ What is the Complement?

لو عندك دالة بوليان **F**، الـ **Complement F'** بيكون ناتج عن قلب كل الـ **0's** إلى **1's** وكل الـ **1's** إلى **0's** في جدول الحقيقة.

Concept	Dual	Complement
Changes	AND ↔ OR , 0 ↔ 1 (not the output)	Only the output values (0 ↔ 1)

1 معنى الـ Complement

لو عندك دالة بوليان F، الـ Complement F' بيكون ناتج عن:

قلب كل الـ 0's إلى 1's وكل الـ 1's إلى 0's في جدول الحقيقة.

الفرق بين الـ Dual والـ Complement:

- **Dual :** بس بيغير بين OR ↔ AND وبين 1 ↔ 0، *مش* الـ output.
- **Complement :** بس بيغير الـ output (1 ↔ 0).

2 طريقتين لإيجاد الـ Complement

1 الطريقة الأولى: نظرية DeMorgan

- خد الدالة الأصلية.
- طبق قانون DeMorgan وكرر التطبيق على كل جزء من الدالة:

$$(AB)' = A' + B'$$
$$(A + B)' = A'B'$$

مثال:

$$F_1 = x'yz' + x'y'z$$

نكمل الـ complement:

$$F_1' = (x'yz' + x'y'z)' = (x'yz')' \cdot (x'y'z)'$$
$$= (x + y' + z)(x + y + z')$$

ده بيورينا النتيجة مباشرة.

2 الطريقة الثانية: Dual + Complement Literals

1. **خد الدالة الأصلية:** $F_1 = x'yz' + x'y'z$
2. **خد الـ Dual:** بدل كل + بـ · (AND) والعكس، وبدل 0 بـ 1 والعكس.
   النتيجة: $(x' + y + z')(x' + y' + z)$
3. **بعد كده اعمل Complement لكل Literal:** بدل كل x بـ x' وكل x' بـ x.

$$F_1' = (x + y' + z)(x + y + z')$$

النتيجة نفس Method 1.

مثال

الدالة:

$$F_2 = x(y'z' + yz)$$

نطبق DeMorgan:

$$F_2' = (x(y'z' + yz))'$$
$$= x' + (y'z' + yz)'$$
$$= x' + (y+z)(y'+z')$$

أو نستخدم Method 2:

- **ال Dual:** $x + (y+z)(y'+z')$
- **Complement لكل Literal:** هيديك نفس النتيجة بالظبط! 😊

8 الجزء الثامن: Canonical & Standard Forms

🏛 الفرق بين Minterms و Maxterms

النوع	التعريف	قاعدة الإشارة	متى تكون 1 أو 0	الترميز
Minterm (m_i)	حاصل ضرب (AND) لكل المتغيرات	المتغير **غير مقلوب لو البت = 1**، مقلوب لو البت = 0	1 = فقط لمجموعة واحدة من القيم	Sum of) **Σ** (minterms
Maxterm (M_i)	حاصل جمع (OR) لكل المتغيرات	المتغير **غير مقلوب لو البت = 0**، مقلوب لو البت = 1	0 = فقط لمجموعة واحدة من القيم	Product of) **Π** (maxterms

⭐ مثال على 3 متغيرات (x, y, z)

Symbol	Maxterm	Symbol	Minterm	z	y	x
M_0	$x+y+z$	m_0	$x'y'z'$	0	0	0
M_1	$x+y+z'$	m_1	$x'y'z$	1	0	0
M_2	$x+y'+z$	m_2	$x'yz'$	0	1	0
M_3	$x+y'+z'$	m_3	$x'yz$	1	1	0
M_4	$x'+y+z$	m_4	$xy'z'$	0	0	1
M_5	$x'+y+z'$	m_5	$xy'z$	1	0	1
M_6	$x'+y'+z$	m_6	xyz'	0	1	1
M_7	$x'+y'+z'$	m_7	xyz	1	1	1

🔑 الفكره:

كل **Minterm = 1** لمجموعة واحدة من القيم.

وكل **Maxterm = 0** لمجموعة واحدة من القيم.

9 Part 9: Expressing Functions in Canonical Forms

Sum of Minterms Σ (F = 1 rows)

كل صف في الجدول تكون فيه الدالة = **1** ← ده اسمه **minterm**. لما نكتب كل المينترمز بـ **OR** ← يبقى عندنا **Sum of Minterms**.

🧪 Example: $F = A + B'C$

A	B	C	B'C	F = A + B'C
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	0	1
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1

الصفوف اللي **F = 1** ← الصفوف: **1,4,5,6,7**

$$F(A,B,C) = \Sigma(1,4,5,6,7)$$

Product of Maxterms Π (F = 0 rows)

كل صف في الجدول تكون فيه الدالة = **0** ← اسمه **maxterm**. لما نكتب كل الماكسترمز بـ **AND** ← يبقى عندنا **Product of Maxterms**.

من نفس المثال – الصفوف اللي **F = 0** ← الصفوف: **0, 2, 3**

$$F(A,B,C) = \Pi(0,2,3)$$

↔ **Quick Reminder:**
Σ → Sum of minterms → OR for F=1 rows
Π → Product of maxterms → AND for F=0 rows

1 0 Part 10: Other Logic Operations

Functions of Two Variables

لما يكون عندنا **2** متغير ثنائيين **(x, y)**، عدد الدوال اللي ممكن تتكون = $2^{2^2} = 16$ دالة.

Function	Expression	Name	Comment
F_0	0	Null	Binary constant 0
F_1	$x \cdot y$	AND	x and y
F_2	$x \cdot y'$	Inhibition	x but not y
F_3	x	Transfer	x
F_4	$x' \cdot y$	Inhibition	y but not x

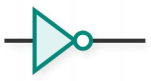
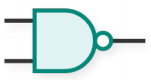
F_5	y	Transfer	y
F_6	$xy' + x'y$	XOR	x or y, not both
F_7	$x + y$	OR	x or y
F_8	$(x + y)'$	NOR	Not-OR
F_9	$xy + x'y'$	XNOR / Equivalence	x equals y
F_{10}	y'	Complement	Not y
F_{11}	$x + y'$	Implication	If y then x
F_{12}	x'	Complement	Not x
F_{13}	$x' + y$	Implication	If x then y
F_{14}	$(x \cdot y)'$	NAND	Not-AND
F_{15}	1	Identity	Binary constant 1

Idea

- **AND / OR / NOT** → Fundamental
- **Buffer / Transfer** → Just copy x or y
- **NAND / NOR** → Complement of AND / OR
- **XOR / XNOR** → "Difference / Equality" between x and y

Part 11: Digital Logic Gates

The 8 Logic Gates

Gate	Symbol	Boolean Expression	Truth Table (00 → 01 → 10 → 11)	Notes
AND		$F = X \cdot Y$	0 0 0 1	Both must be 1
OR		$F = X + Y$	0 1 1 1	At least one 1
NOT		$F = X'$	1 0 (1 input)	Inverts input
Buffer		$F = X$	0 1 (1 input)	Passes input unchanged
NAND		$F = (X \cdot Y)'$	1 1 1 0	Output 0 only when both = 1
NOR		$F = (X + Y)'$	1 0 0 0	Output 1 only when both = 0
XOR		$X \oplus Y = X'Y + XY'$	0 1 1 0	Different inputs → 1
XNOR		$(X \oplus Y)' = XY + X'Y'$	1 0 0 1	Equal inputs → 1

Important Property: NAND & NOR

- **Commutative:** $X \uparrow Y = Y \uparrow X$ ✅ **(Order doesn't matter)**
- **NOT Associative:** $(X \uparrow Y) \uparrow Z \neq X \uparrow (Y \uparrow Z)$ ❌ **(Bracket order matters!)**

NOR Non-Associativity Proof

$$(X \downarrow Y) \downarrow Z = (X+Y)' \downarrow Z \neq X \downarrow (Y \downarrow Z)$$

Part 12: Integrated Circuits (ICs)

IC Classifications

الـ **ICs** أو الدوائر المتكاملة عبارة عن شريحة فيها مجموعة من البوابات المنطقية، واللي بيحدد نوع الـ **IC** هو عدد البوابات اللي جواه.

Type	Abbreviation	Gate Count	Examples
Small-Scale Integration	**SSI**	fewer than 10 gates	Basic gates: AND, OR, NOT
Medium-Scale Integration	**MSI**	10 - 1,000 gates	Adders, MUX, Decoders
Large-Scale Integration	**LSI**	Thousands of gates	Processors, Memory chips
Very Large-Scale Integration	**VLSI**	> 100,000 gates	Modern CPU, GPU

كل ما عدد البوابات زاد، الشرائح بقيت أذكى وأقدر تعمل عمليات أكبر.

Positive vs Negative Logic

Positive Logic

- High voltage (H) = **1**
- Low voltage (L) = **0**

Negative Logic

- High voltage (H) = **0**
- Low voltage (L) = **1**

كل حاجة معكوسة

Gate Conversion (Positive → Negative Logic)

Positive Logic	→	Negative Logic
AND	→	OR
OR	→	AND

السبب: ده تطبيق عملي لقانون **DeMorgan**. لو عندك **AND gate** بتطلع **1** لما كل المدخلات **1**، في **Negative Logic** هيطلع **1** لو واحد على الأقل **0**، زي **OR gate**.